

COMP2054-ADE: Introduction to big-Oh

ADE-Lec02a

Lecturer: Andrew Parkes

Email: andrew.parkes@Nottingham.ac.uk

Recap: Removing details

- (For an example program) According to precisely how we count steps we might get one of
 - $5 \log_2(n) + 2$
 - $9 \log_2(n) + 5$, etc.
- Also, this counts “steps”
 - the translation to runtime depends on the compiler, hardware, etc.
- **Need a way to suppress such ‘implementation-dependent details’ and describe the ‘scaling as n increases’, and that is mathematically well-defined**

Aim: Classification of Functions

- CS needs a way to group together **functions** by their scaling behaviour as n increases, and the classification should
 - Remove unnecessary details
 - Be (relatively) quick and easy
 - Handle 'weird' functions that can happen for runtimes
 - Still be mathematically well-defined
- Experience of CS is that this is best done by the "big-Oh notation and family":

**O (Big-Oh), Ω (Big-Omega), Θ (Big-Theta),
 o (little-oh) and ω (little-omega)**

“Surprising”/“Advanced” uses of Big-Oh family

- Stirling’s approximation (which we use later)
https://en.wikipedia.org/wiki/Stirling%27s_approximation
$$\ln(n!) = n \ln n - n + O(\ln n)$$
 - where “ln” is the natural logarithm “log base e”
 - (Note there is no reference to an algorithm!)
- Polynomial functions are $n^{O(1)}$
- The “best case of algorithm X is $O(n \log n)$ ”
- Does there exist an algorithm for TSP with runtime $2^{o(n)}$?

Need a good/proper understanding of Big-Oh family to interpret the above

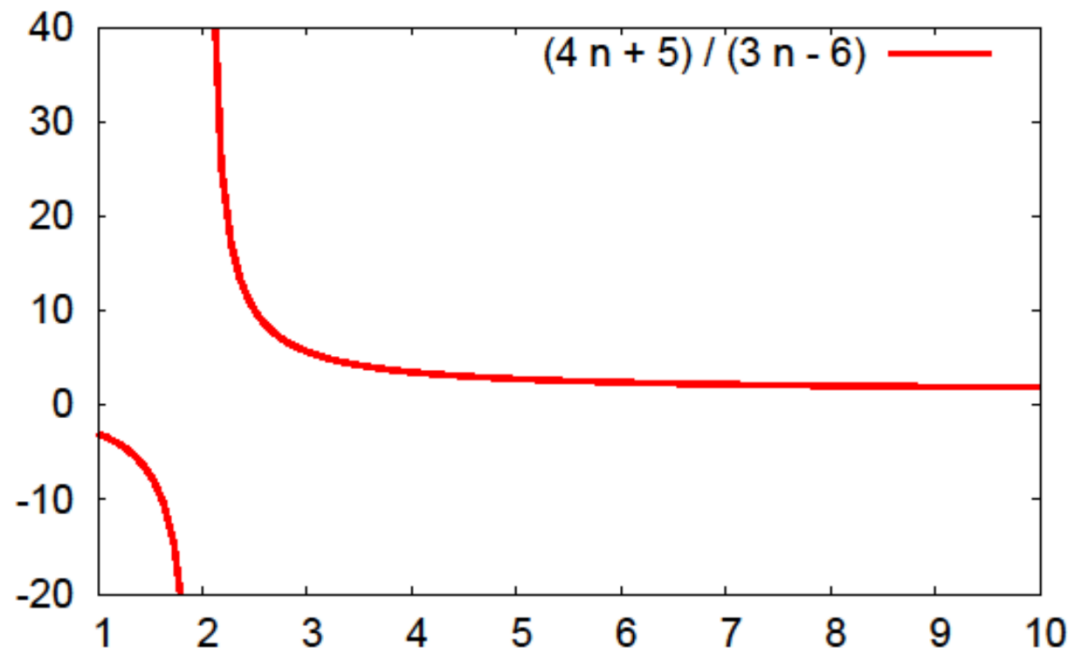
Big-Oh Notation: Motivations

- Suppose we have two functions
 1. $4n + 5$
 2. $3n - 6$
- We want to express that the behaviour is driven by fact that both are linear in n , and to suppress the details of the exact function
- One way (not the only way!) to motivate definitions is to look at their ratio:

Important (Subtle?) Comment

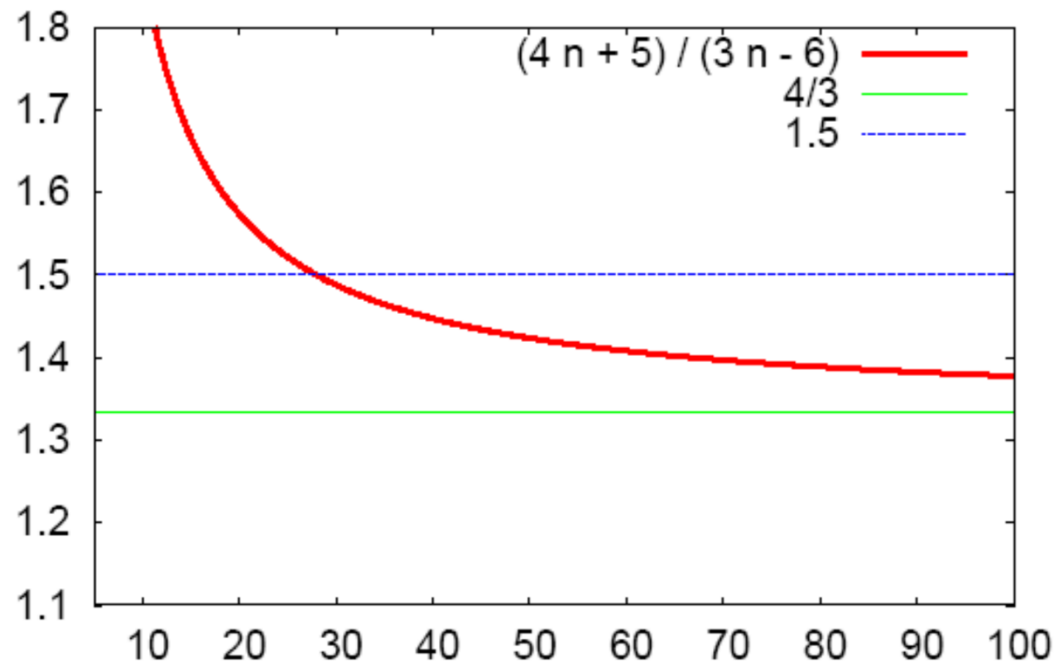
- Note that at this point we are just talking about functions, “ $f(n)$ ”, of a single parameter n
- Big-Oh is often applied to runtimes, but this is not essential
 - The big-Oh definitions are just in terms of functions
 - It is not only for “worst case runtime” – definitions do not even refer to algorithms
 - But the big-Oh definition should be “designed” to be suitable for discussion of “runtime functions”
 - How do we motivate a good design of the definitions?

Ratio of two linear functions .1



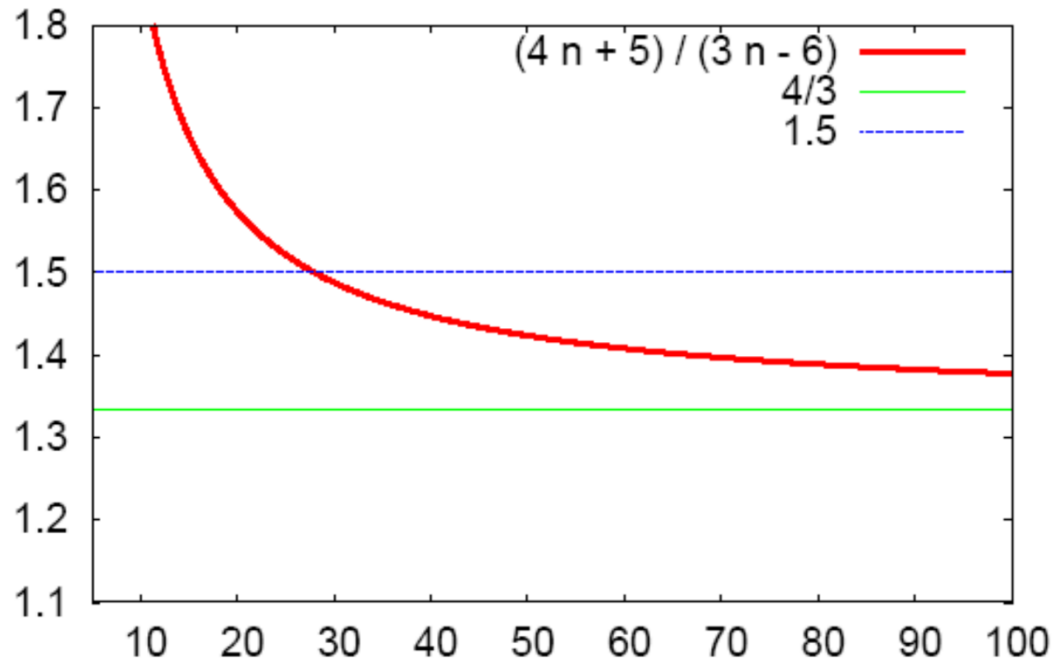
- The behaviour at small n is messy

Ratio of two linear functions .2



- At larger values of n the ratio starts to behave more predictably
 - Suggests definitions should use “forall $n \geq \dots$ ”

Ratio of two linear functions .3



- It looks like:

$$(4n+5) \leq 1.5 * (3n-6) \text{ for all } n \geq 30 \quad ?$$

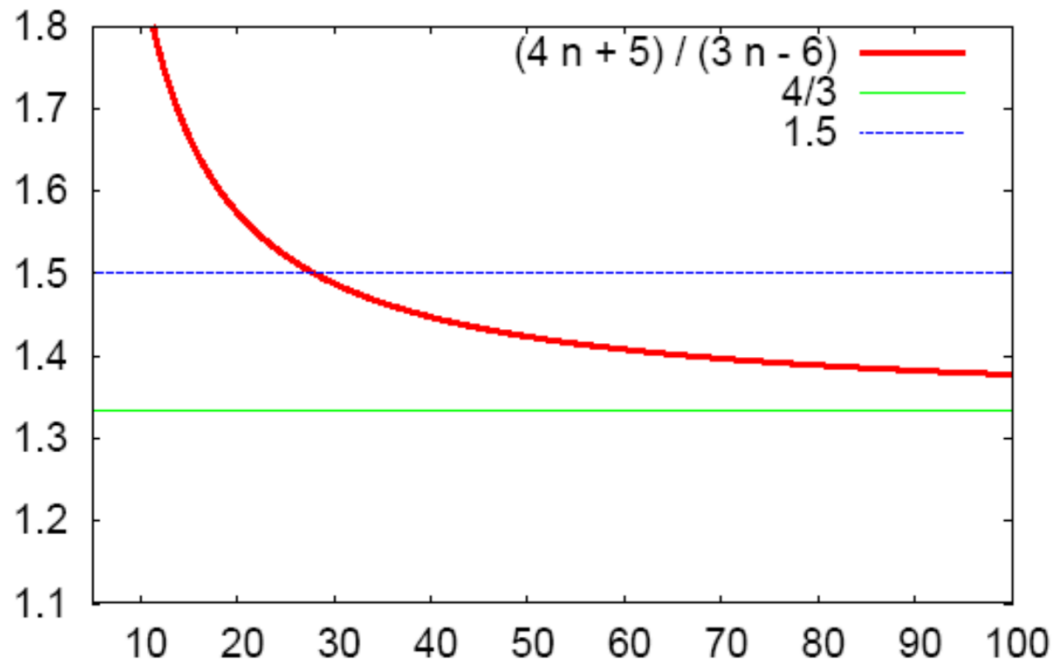
Exercise: computation

- For precisely what values of n is the following true:

$$(4n+5) \leq 1.5 * (3n-6)$$

- Answer:
 - $8n + 10 \leq 9n - 18$ (by x2)
 - $28 \leq n$ (add $18-8n$ to both sides)

Ratio of two linear functions



- Hence we have:
 $(4n+5) \leq 1.5 * (3n-6)$ for all $n \geq 28$
- Suggests definitions that place upper bounds on the function relative to another function:

*** Definition: "Big-Oh" ***

For two positive functions $f(n)$ and $g(n)$, of a single variable n , then

$f(n)$ is $O(g(n))$ "f is Big-Oh of g"

if and only if there exist constants $c \geq 0$ and $n_0 \geq 1$ such that

for all $n \geq n_0$. $f(n) \leq c g(n)$

Which functions are used?

- Big Oh is intended to functions of **positive integers** (size), and that are **positive real values** because they (often) represent runtimes:
 - $f : \mathbb{N}^+ \rightarrow \mathbb{R}^+$ (and similarly for g)
 - Where $\mathbb{N}^+ = \{ 1, 2, 3, \dots \}$ and
 - $\mathbb{R}^+ = \{ x \in \mathbb{R} \mid x \geq 0 \}$
- That is, assume, $f(n) \geq 0 \ \forall n \geq 1$
- Sometimes, for convenience, might relax to
 - $f(n) \geq 0 \ \forall n \geq N$ for some constant N

Big-Oh Defn. : “EEA”

For two positive functions $f(n)$ and $g(n)$, of a single variable n , then

$f(n)$ is $O(g(n))$ “ f is Big-Oh of g ”

*if and only if there exist constants $c \geq 0$ and $n_0 \geq 1$ such that
for all $n \geq n_0$, $f(n) \leq c g(n)$*

Quantifier and logical structure:

“ f is $O(g)$ ” $\leftrightarrow$

$\exists c. \exists n_0. [\forall n \geq n_0. (f(n) \leq c g(n))]$

Common error is to accidentally change the
quantifier order – it must **$\exists \exists \forall$**

With abbrevs: iff=“if and only if”, s.t. “such that”

f is $O(g)$ iff $\exists c. \exists n_0$. s.t. $\forall n \geq n_0. (f(n) \leq c g(n))$]

Big-Oh Defn. : “c not c(n)”

f(n) is O(g(n)) iff

there exist constants $c \geq 0$ and $n_0 \geq 1$ s.t. for all $n \geq n_0$, $f(n) \leq c g(n)$

VITAL: quantifier structure is “pick c before handling the for all n”

- “c” must be independent of n;
- **We are not allowed to satisfy the inequality by allowing c to depend on n**

Rationale:

If we allowed c(n), then we could take $c(n) = g(n)/f(n)$. Then the inequality would be satisfied, and the definition would tell us nothing useful about the relative growth rates of f and g.

Big-Oh Defn. : “for all n”

$f(n)$ is $O(g(n))$ iff

there exist constants $c \geq 0$ and $n_0 \geq 1$ s.t. for all $n \geq n_0$, $f(n) \leq c g(n)$

VITAL: the definition requires proving for all $n \geq n_0$, $f(n) \leq c g(n)$

It is not sufficient to simply show for $n = n_0$, $f(n) \leq c g(n)$

Obviously, this would be useless as a definition; as could just always pick c
This is a common error in tests (!).

How do formally prove statements of “for all n”?

- In worst case, or formally, would need induction.
- In general we just use “obvious properties” such as “for all n , $2n \geq n$ ”

Big-Oh Defn. : “Scope”

$f(n)$ is $O(g(n))$ iff

there exist constants $c \geq 0$ and $n_0 \geq 1$ s.t. for all $n \geq n_0$, $f(n) \leq c g(n)$

VITAL: Note that words such as “algorithm” or “worst case runtime” do not appear in the definition!

It only mentions functions!!

Usually, we will use it for functions, f , that are something to do with the runtime of algorithms.

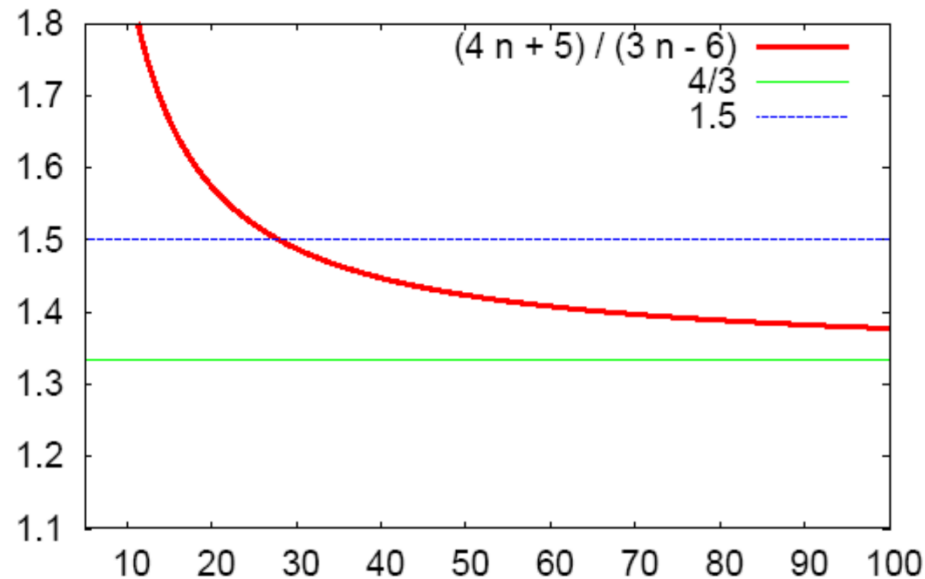
Convention is that, if nothing else is specified then we would taking f to be worst-case runtime of an algorithm.

But it could equally well be used for

- Best-case runtime
- Worst-case space usage, etc. etc.

Recap:

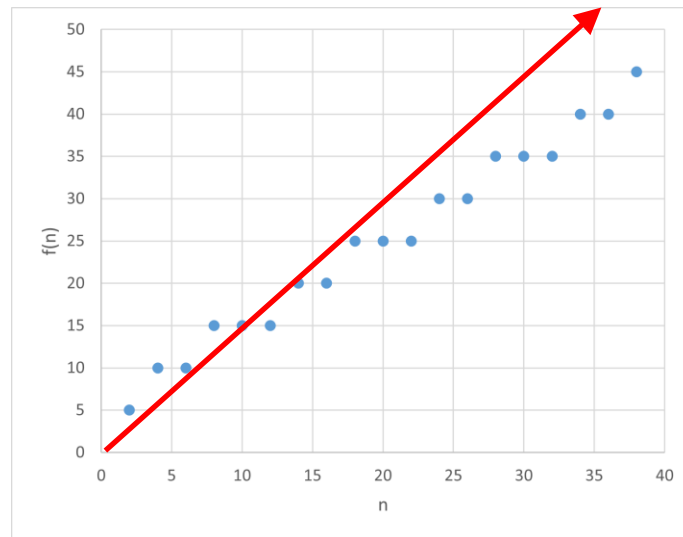
Ratio of two linear functions



- We found: $(4n+5) \leq 1.5 * (3n-6)$ for all $n \geq 28$
- So can now say: $(4n+5)$ is $O((3n-6))$
 - (And the proof would use $c=1.5$ and $n_0=28$)

Example: Function with steps

- Consider a function that is more irregular:



- It looks linear, but how do we formalise this?
- Show it is bounded above by some linear function, $c \cdot n$, above some n_0 . E.g. pick $1.5n$
- Which shows $f(n)$ is $O(n)$ – using $g(n)=n$ and $c=1.5$, $n_0=15$

EXERCISE.1

$f(n)$ is $O(g(n))$ iff $\exists c, n_0$ s.t.
 $\forall n \geq n_0. f(n) \leq c g(n)$

- Show that the function:

$$f(n) = 1$$

is $O(1)$

ANS: (pause and try!)

EXERCISE.1a

- Show that the function:

$$f(n) = 1$$

is $O(1)$

ANS: pick c, n_0 s.t. $1 \leq c \leq 1$ for all $n \geq n_0$
 $c=1$ $n_0=1$ Done.

$c=3499993$, $n_0=392875$

Proof still works. **Definition does not say pick smallest c, n_0 .**

EXERCISE.2

- Can we show that the function:

$$f(n) = 2$$

is $O(1)$?

ANS: (pause and try!)

EXERCISE.2a

- Can we show that the function:

$$f(n) = 2$$

is $O(1)$?

ANS: need to

pick c, n_0 s.t. $2 \leq c \cdot 1$ for all $n \geq n_0$

$c=2$ $n_0=1$

DONE

EXERCISE.3

- Show that the function:

$$f(n) = k$$

is $O(1)$ for any fixed constant k

ANS: pick c, n_0 . $k \leq c \cdot 1$

$c = 2 \cdot \max(k, 1)$ is allowed (not using n)

$n_0 = 2$

$c = k/2$ fails – but does not matter as need
“exists c ” not “for all c ”

EXERCISE.4

- Can we show that the function:

$$f(n) = 1$$

is $O(n)$

ANS: (pause and try!)

EXERCISE.4a

- Can we show that the function:

$$f(n) = 1$$

is $O(n)$

ANS: pick c, n_0 . s.t. $1 \leq c n$ for all $n \geq n_0$
 $n_0=1$

$c=1$ need $1 \leq n$ for all $n \geq 1$

DONE:

i.e. 1 is $O(n)$

This might “seem wrong”? But it is correct “from the definition”. It is also true “ 1 is $O(1)$ ”.

See later discussions.

EXERCISE.5

- Can we show that the function:
$$f(n) = k$$

is $O(n)$ for any fixed constant $k \geq 0$

ANS:

EXERCISE.5a

- Show that the function:

$$f(n) = k$$

is $O(n)$ for any fixed constant $k \geq 0$

ANS: want

$$k \leq c n \quad \text{for } n \geq n_0$$

Note that $c=k$, $n_0=1$ will suffice

Hence, k is $O(n)$, as well as $O(1)$.

EXERCISE.6

- Show that the function:

$$f(n) = k n$$

is $O(n)$ for any fixed constant k

ANS: (pause and try!)

EXERCISE.7

- Show that the function:

$$f(n) = n + k$$

is $O(n)$ for any fixed constant k

ANS: pick c, n_0 s.t

$$n+k \leq c n \quad \text{for all } n \geq n_0$$

Pick $c=1$ $n+k \leq n$ fails (for $k>0$)

$c=2$ $k \leq n$ pick $n_0=k$ DONE

or

$c = (1+k)$, then $n+k \leq (1+k)n$ pick $n_0=1$
DONE (Exercise: check this is correct)

EXERCISE.8

- Show that the function:

$$f(n) = n$$

is **not** $O(1)$.

ANS: (pause and try)

EXERCISE.8a

- Show that the function:

$$f(n) = n$$

is **not** $O(1)$.

ANS: can we pick? c, n_0 s.t.

$$n \leq c \quad \text{for all } n \geq n_0$$

Pick $c = n$? NO!!!!!!

CANNOT pick a c . ($c = \infty$ is not allowed)

Hence n is not $O(1)$.

(It is also $O(n^2)$, but $O(n)$ is “best”)

EXERCISE.8a (cont)

- Show that the function:

$$f(n) = n$$

is **not** $O(1)$.

ANS: We would need to provide **constants** c and n_0 (c cannot depend on n) and prove

$$n \leq c \cdot 1 \quad \forall n \geq n_0$$

But this is clearly impossible.

E.g. take $n = \max(c+1, n_0)$.

BREAK until next lecture lec02b